Assignment 1

1. Requirements

The assignment requires students to design and implement (or enhance) a testing technique using AI methods (e.g., LLMs). The students can either **choose static testing, black-box dynamic testing or white-box dynamic testing techniques.**

Note: This page shows the list of tools for static code analysis; The black-box testing techniques include Equivalence Partitioning (EP), Boundary Value Analysis (BVA), Testing Combinations of Inputs, State Transition Testing Model Generator, Decision Table Testing; The white-box testing can measure the statement coverage, branch coverage, condition coverage, path coverage, d-u coverage, etc.

Now the assignment requires students to implemented as a tool where it can take either two forms of input: (1) the requirements of a system; (2) the testing codebase. The tool is expected to analyze either (1) the requirements of a system; (2) the testing codebase (or a partial module) to create test cases.

2. Submission Artifact

1. Input: requirement/project code base
2. Tool artifact: prompts used, model used, model-generated code
3. Generated output: reported alarms (for static analysis); test cases (for black- and white-box analysis)
4. Experimental Analysis (Accuracy, Coverage, Generalizability, etc)
5. Project report
    a. Comparison to traditional non-AI-based technique, pros and cons
    b. Analytical report: How you encounter limitations of AI and how you improve the tool while practicing?
    c. Summary

3. Assessment Criteria

- Understanding of concepts 10%
- Coherence of design and implementation 20%
- Coverage and effectiveness/usefulness 40%
- In-depth analysis (generalizability demonstration, reasoning, etc) 20%
- Presentation 10%

2. Presentation

Each group has **15 minutes** to complete the project presentation in English, covering all aspects listed above. A Q&A session follows. Reviewers will ask questions based on the submitted documents, presentation content, and the software testing fundamentals that students are expected to have learned.

The contribution allocation among group members is equal by default. Any deviation must be separately applied for and approved with the signatures of all members.

3. Submission

Each group must submit the following materials by email to the TA **one day before the presentation date**:

a) **Submission Artifact**: Contains all required content described above (the cover page must include the team ID, full names of all members, and student IDs).

b) **Final Presentation PPT**: The first slide must include the team ID and full names and student IDs of all members.

The report and PPT must be submitted in **PDF format**, and the test scripts must be submitted as a compressed file.

Submission and Presentation Schedule:

- **Submission Deadline**: Week 8 (April 20th), Monday, before 17:00 pm
- **Presentation Dates**: Week 8–9, Tuesday/Thursday, 10:00–11:35

Example Submission Ex1:

Title: LLM-based Dynamic Black-box Testing for Multi-Item Smart Vending Machine

1. Input:

System Overview: The system under test is a smart vending machine deployed in a public area (e.g., subway station). The internal software, hardware control logic, and database are not visible to the tester. Testing is performed only from the externally observable behavior, based on the provided requirements.

Functional Requirements:

- R1. Item Selection. The vending machine offers three categories of products: Drinks (Price range: $1.50, $3.00); Snacks (Price range: $2.00, $4.50), Hot food (Price range: $5.00, $10.00).
- R2. Accepted Payment Methods. The vending machine accepts multiple payment methods: 1. Coins: $0.10, $0.25, $0.50, $1.00; 2. Banknotes: $5.00, $10.00.
- R3. Payment Constraints. The total inserted amount must be greater than or equal to the item price. The machine returns change only up to $5.00; rejects transactions requiring more than $5.00 change.
- R4. Inventory Constraints. An item may become out of stock during payment.

2. Tool Artifact:

LLM Used: GPT-4o
Prompt Used:

You are a software testing assistant. Given the following system overview and requirement, identify:
1) Input variables 2) Equivalence partitions (valid and invalid) 3) Boundary values 4) Concrete test cases
Requirement: {requirement_text}

3. Generated Output

Equivalence Partitioning:

ID	Description	Outcome
EP1	Inserted amount < item price	valid

EP2	Inserted amount > item price	invalid
…		

Boundary Value Analysis:

Boundary	Values
Item price	$1.4, $1.5, $1.6
Payment Total	$0, $1.4, $15.1
…	

Sample Test Cases:

Test Case	Scenario	Expected Result
TC1	Snack $3.00, paid $8.50	Reject (change > $5)
TC2	Snack $3.00, paid $3.50	Payment Success, return change $0.50
…		

4. Experimental Analysis

4.1 Coverage of EP/BVA and test cases.

4.2 Missing item analsysis.

4.3 Refining Prompts for improving coverage, accuracy, and generalizability.

5. Project Report

5.1 Comparison to traditional non-AI-based technique, pros and cons

5.2 Analytical report: limitations of AI and the ways to improve the tool

5.3 Summary

Example Submission Ex2:

Title: LLM-based Static Analysis

1. Input:

System Overview:

Axios is a promise-based network request library that can run in both Node.js and browsers. This library is adapted from the original Axios v1.3.4 version to make it compatible with OpenHarmony, while retaining its existing usage patterns and features.

- HTTP requests
- Promise API
- Request and response interceptors
- Transformation of request and response data
- Automatic conversion of JSON data

The source code of Axios can be obtained at
https://ohpm.openharmony.cn/#/cn/detail/@ohos%2Faxios

List of interfaces and attributes

List of interfaces

Interface	Parameters	Function
axios(config)	config: request configuration	Send a request
Axios.create(config)	config: request configuration	Create an instance
Axios.request(config)	config: request configuration	Send a request
Axios.get(url[, config])	url: request address config: request configuration	Send the get request
axios.delete(url[, config])	url: request address config: request configuration	Send a delete request
axios.post(url[, data[, config]])	url: request address data: send request body data config: request configuration	Send a post request
Axios.put(url[, data[, config]])	url: request address data: send request body data config: request configuration	Send a put request

List of attributes

Attributes	Describe
Axios.defaults['xxx']	Default setting. The value is the configuration item in the request configuration config. For example, axios.defaults.headers to get the header information
Axios.interceptors	Interceptor. Reference The use of interceptor

2. Tool Artifact

Example Prompt:

You are a static code analyzer. Analyze the following {language} code and detect potential issues.
Identify the following:
1. Syntax errors
2. Security vulnerabilities
3. Deprecated or incompatible API usage
4. Potential runtime errors
5. Code quality issues
Return the results in structured JSON format:

```
{
  "issues": [
    {
      "line": <line_number>,
      "type": "<issue_type>",
      "description": "<detailed description>",
      "severity": "<low/medium/high>"
    }
  ]
}
```

Code:
{source_code}

3. Generated Output

Issue 1:


```
[{
    "line": 4,
    "type": "Resource Management",
    "description": "File opened using 'open' but not managed with a context manager. If an exception occurs, the file may remain open.",
    "severity": "medium",
    "recommendation": "Use 'with open(filename, 'r') as f:' to automatically close the file.",
    "category": "Code Quality",
```

"reference": "https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files"
}]

Test Case (Proof of concept):

```python
import os

def test_resource_management_leak():
    filename = "testfile.txt"

    # Prepare test file
    with open(filename, 'w') as f:
        f.write("Hello World!")

    # Problematic function (from analyzer report)
    def read_file_bad(filename):
        f = open(filename, 'r')
        data = f.read()
        # Simulate exception occurring before closing
        raise Exception("Simulated error")
        f.close()
        return data

    # Attempt to call read_file_bad and catch exception
    try:
        read_file_bad(filename)
    except Exception as e:
        print(f"Caught exception: {e}")

    # Check if file is still open (file descriptor leak)
    # On Unix, you can check open file descriptors
    # For simplicity, we'll just try to delete the file; if OS prevents deletion, it indicates file is open
    try:
        os.remove(filename)
        print("File successfully deleted (no leak detected)")
    except PermissionError:
        print("File deletion failed: file may still be open (resource leak)")
```

4. Experimental Analysis

4.1 False Alarms Analysis

4.2 Refining Prompts for improving accuracy

4.3 Try more projects to improve generalizability

4.4 Bug Reporting and Validation by the developers

5. Project Report

5.4 Comparison to traditional non-AI-based technique, pros and cons

5.5 Analytical report: limitations of AI and the ways to improve the tool

5.6 Summary

Example Submission Ex3

Title: LLM-based White-box Testing

1. Input

System Overview:

Axios is a promise-based network request library that can run in both Node.js and browsers. This library is adapted from the original Axios v1.3.4 version to make it compatible with OpenHarmony, while retaining its existing usage patterns and features.

- HTTP requests
- Promise API
- Request and response interceptors
- Transformation of request and response data
- Automatic conversion of JSON data

The source code of Axios can be obtained at
https://ohpm.openharmony.cn/#/cn/detail/@ohos%2Faxios

List of interfaces and attributes

List of interfaces

Interface	Parameters	Function
axios(config)	config: request configuration	Send a request
Axios.create(config)	config: request configuration	Create an instance
Axios.request(config)	config: request configuration	Send a request
Axios.get(url[, config])	url: request address config: request configuration	Send the get request
axios.delete(url[, config])	url: request address config: request configuration	Send a delete request
axios.post(url[, data[, config]])	url: request address data: send request body data config: request configuration	Send a post request
Axios.put(url[, data[, config]])	url: request address data: send request body data config: request configuration	Send a put request

List of attributes

Attributes	Describe
Axios.defaults['xxx']	Default setting. The value is the configuration item in the request configuration config. For example, axios.defaults.headers to get the header information
Axios.interceptors	Interceptor. Reference The use of interceptor

2. Tool Artifact

Prompt:

You are an expert software tester and white-box testing assistant.
*Your task is to analyze the following {language} function/module and generate a set of test cases that achieve **full statement coverage**.*

Please do the following:

1. Identify all executable statements in the code.
2. For each statement, generate test input values that will execute it at least once.
*3. Output the test cases in a **structured JSON format** suitable for automated testing.*

Structured JSON format:

```
{
  "function": "<function_name>",
  "test_cases": [
    {
      "input": { "<parameter_name>": <value>, ... },
      "expected_output": "<expected output or behavior>",
```

"covered_statements": [<line_numbers>],
"notes": "<optional explanation>"
}
]
}

Code to analyze:
{code_snippet}

Notes:
- Include edge cases if necessary to cover all branches of conditional statements.
- If a statement is unreachable due to a logic error, mark it as "unreachable".
- Provide explanations for how each test case achieves statement coverage.

3. Generated Output

Tested Function:

```
 * Clean up temporary directory files
 * @param cacheDir: temporary directory
 * @param options: request configuration
 */
const deleteFile = (cacheDir:string, options:request.UploadConfig):void => {
    const axios_temp = 'axios_temp'
    let path_temp = `${cacheDir}/${axios_temp}/`;
    try {
        if (options) {
            //Delete specified file
            options && options.files.forEach(item => {
                item.uri && fs.unlink(item.uri.replace(`internal://cache/${axios_temp}/`, path_temp));
            })
        } else {
            // Delete files that exceed 12 hours
            let filenames = fs.listFileSync(path_temp);
            let now = Date.now();
            for (let i = 0; i < filenames.length; i++) {
                let path = path_temp + filenames[i];
                let stat = fs.statSync(path);
                if (now - stat.atime * 1000 >= 1 * 60 * 60 * 1000) {
                    fs.unlink(path);
                }
            }
        }
    }
    catch (err) {
        hilog.error(DOMAIN, TAG, `delete file failed with error message message: ${err.message}, code: ${err.code}`);
    }
}
```

Test Cases:

TestCase1: Delete Specified Files

```
deleteFile("/tmp/cache", {
    files: [
        { uri: "internal://cache/axios_temp/file1.txt" },
        { uri: "internal://cache/axios_temp/file2.txt" }
    ]
});
```

TestCase2: Exception Handling

```
fs.unlink.mockImplementation(() => { throw { message: "permission denied", code: 13 } });

deleteFile("/tmp/cache", {
    files: [{ uri: "internal://cache/axios_temp/file3.txt" }]
});
```

4. Experimental Analysis

4.1 False Alarms Analysis

4.2 Refining Prompts for improving accuracy

4.3 Try more projects to improve generalizability

4.4 Bug Reporting and Validation by the developers

5. Project Report

5.7 Comparison to traditional non-AI-based technique, pros and cons

5.8 Analytical report: limitations of AI and the ways to improve the tool

5.9 Summary